8.3 Implementing Trees

The Tree and BinaryTree classes that we have defined thus far in this chapter are both formally ***abstract base classes***. Although they provide a great deal of support, neither of them can be directly instantiated. We have not yet defined key implementation details for how a tree will be represented internally, and how we can effectively navigate between parents and children. Specifically, a concrete implementation of a tree must provide methods root, parent, num_children, children, __len__, and in the case of BinaryTree, the additional accessors left and right.

There are several choices for the internal representation of trees. We describe the most common representations in this section. We begin with the case of a ***binary tree***, since its shape is more narrowly defined.

8.3.1 Linked Structure for Binary Trees

A natural way to realize a binary tree T is to use a ***linked structure***, with a node (see Figure 8.11a) that maintains references to the element stored at a position p and to the nodes associated with the children and parent of p. If p is the root of T, then the parent field of p is None. Likewise, if p does not have a left child (respectively, right child), the associated field is None. The tree itself maintains an instance variable storing a reference to the root node (if any), and a variable, called size, that represents the overall number of nodes of T. We show such a linked structure representation of a binary tree in Figure 8.11b.

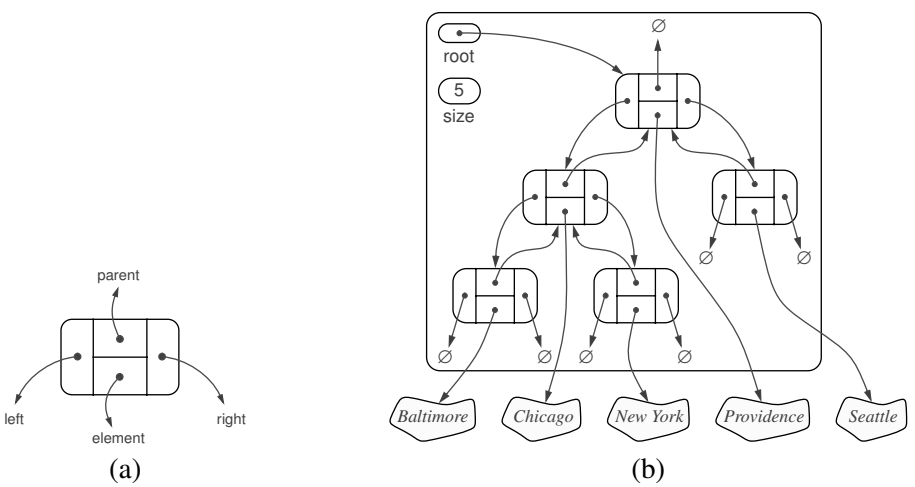


Figure 8.11: A linked structure for representing: (a) a single node; (b) a binary tree.

Python Implementation of a Linked Binary Tree Structure

In this section, we define a concrete LinkedBinaryTree class that implements the binary tree ADT by subclassing the BinaryTree class. Our general approach is very similar to what we used when developing the PositionalList in Section 7.4: We define a simple, nonpublic _Node class to represent a node, and a public Position class that wraps a node. We provide a _validate utility for robustly checking the validity of a given position instance when unwrapping it, and a _make_position utility for wrapping a node as a position to return to a caller.

Those definitions are provided in Code Fragment 8.8. As a formality, the new Position class is declared to inherit immediately from BinaryTree.Position. Technically, the BinaryTree class definition (see Code Fragment 8.7) does not formally declare such a nested class; it trivially inherits it from Tree.Position. A minor benefit from this design is that our position class inherits the __ne__ special method so that syntax p != q is derived appropriately relative to __eq__.

Our class definition continues, in Code Fragment 8.9, with a constructor and with concrete implementations for the methods that remain abstract in the Tree and BinaryTree classes. The constructor creates an empty tree by initializing _root to None and _size to zero. These accessor methods are implemented with careful use of the _validate and _make_position utilities to safeguard against boundary cases.

Operations for Updating a Linked Binary Tree

Thus far, we have provided functionality for examining an existing binary tree. However, the constructor for our LinkedBinaryTree class results in an empty tree and we have not provided any means for changing the structure or content of a tree.

We chose not to declare update methods as part of the Tree or BinaryTree abstract base classes for several reasons. First, although the principle of encapsulation suggests that the outward behaviors of a class need not depend on the internal representation, the *efficiency* of the operations depends greatly upon the representation. We prefer to have each concrete implementation of a tree class offer the most suitable options for updating a tree.

The second reason we do not provide update methods in the base class is that we may not want such update methods to be part of a public interface. There are many applications of trees, and some forms of update operations that are suitable for one application may be unacceptable in another. However, if we place an update method in a base class, any class that inherits from that base will inherit the update method. Consider, for example, the possibility of a method T.replace(p, e) that replaces the element stored at position p with another element e. Such a general method may be unacceptable in the context of an ***arithmetic expression tree*** (see Example 8.7 on page 312, and a later case study in Section 8.5), because we may want to enforce that internal nodes store only operators as elements.

For linked binary trees, a reasonable set of update methods to support for general usage are the following:

T.add_root(e): Create a root for an empty tree, storing e as the element, and return the position of that root; an error occurs if the tree is not empty.

T.add_left(p, e): Create a new node storing element e, link the node as the left child of position p, and return the resulting position; an error occurs if p already has a left child.

T.add_right(p, e): Create a new node storing element e, link the node as the right child of position p, and return the resulting position; an error occurs if p already has a right child.

T.replace(p, e): Replace the element stored at position p with element e, and return the previously stored element.

T.delete(p): Remove the node at position p, replacing it with its child, if any, and return the element that had been stored at p; an error occurs if p has two children.

T.attach(p, T1, T2): Attach the internal structure of trees T1 and T2, respectively, as the left and right subtrees of leaf position p of T, and reset T1 and T2 to empty trees; an error condition occurs if p is not a leaf.

We have specifically chosen this collection of operations because each can be implemented in $O(1)$ worst-case time with our linked representation. The most complex of these are delete and attach, due to the case analyses involving the various parent-child relationships and boundary conditions, yet there remains only a constant number of operations to perform. (The implementation of both methods could be greatly simplified if we used a tree representation with a sentinel node, akin to our treatment of positional lists; see Exercise C-8.40).

To avoid the problem of undesirable update methods being inherited by subclasses of LinkedBinaryTree, we have chosen an implementation in which none of the above methods are publicly supported. Instead, we provide *nonpublic* versions of each, for example, providing the underscored _delete in lieu of a public delete. Our implementations of these six update methods are provided in Code Fragments 8.10 and 8.11.

In particular applications, subclasses of LinkedBinaryTree can invoke the nonpublic methods internally, while preserving a public interface that is appropriate for the application. A subclass may also choose to wrap one or more of the nonpublic update methods with a public method to expose it to the user. We leave as an exercise (R-8.15), the task of defining a MutableLinkedBinaryTree subclass that provides public methods wrapping each of these six update methods.

```
class LinkedBinaryTree(BinaryTree):
  """Linked representation of a binary tree structure."""

  class _Node:         # Lightweight, nonpublic class for storing a node.
    __slots__ = '_element', '_parent', '_left', '_right'
    def __init__(self, element, parent=None, left=None, right=None):
      self._element = element
      self._parent = parent
      self._left = left
      self._right = right

  class Position(BinaryTree.Position):
    """An abstraction representing the location of a single element."""

    def __init__(self, container, node):
      """Constructor should not be invoked by user."""
      self._container = container
      self._node = node

    def element(self):
      """Return the element stored at this Position."""
      return self._node._element

    def __eq__(self, other):
      """Return True if other is a Position representing the same location."""
      return type(other) is type(self) and other._node is self._node

  def _validate(self, p):
    """Return associated node, if position is valid."""
    if not isinstance(p, self.Position):
      raise TypeError('p must be proper Position type')
    if p._container is not self:
      raise ValueError('p does not belong to this container')
    if p._node._parent is p._node:          # convention for deprecated nodes
      raise ValueError('p is no longer valid')
    return p._node

  def _make_position(self, node):
    """Return Position instance for given node (or None if no node)."""
    return self.Position(self, node) if node is not None else None
```

Code Fragment 8.8: The beginning of our LinkedBinaryTree class (continued in Code Fragments 8.9 through 8.11).

```
41    #------------------------- binary tree constructor -------------------------
42    def __init__(self):
43      """Create an initially empty binary tree."""
44      self._root = None
45      self._size = 0
46
47    #------------------------- public accessors -------------------------
48    def __len__(self):
49      """Return the total number of elements in the tree."""
50      return self._size
51
52    def root(self):
53      """Return the root Position of the tree (or None if tree is empty)."""
54      return self._make_position(self._root)
55
56    def parent(self, p):
57      """Return the Position of p's parent (or None if p is root)."""
58      node = self._validate(p)
59      return self._make_position(node._parent)
60
61    def left(self, p):
62      """Return the Position of p's left child (or None if no left child)."""
63      node = self._validate(p)
64      return self._make_position(node._left)
65
66    def right(self, p):
67      """Return the Position of p's right child (or None if no right child)."""
68      node = self._validate(p)
69      return self._make_position(node._right)
70
71    def num_children(self, p):
72      """Return the number of children of Position p."""
73      node = self._validate(p)
74      count = 0
75      if node._left is not None:          # left child exists
76        count += 1
77      if node._right is not None:         # right child exists
78        count += 1
79      return count
```

Code Fragment 8.9: Public accessors for our LinkedBinaryTree class. The class begins in Code Fragment 8.8 and continues in Code Fragments 8.10 and 8.11.

```
80    def _add_root(self, e):
81      """Place element e at the root of an empty tree and return new Position.
82
83      Raise ValueError if tree nonempty.
84      """
85      if self._root is not None: raise ValueError('Root exists')
86      self._size = 1
87      self._root = self._Node(e)
88      return self._make_position(self._root)
89
90    def _add_left(self, p, e):
91      """Create a new left child for Position p, storing element e.
92
93      Return the Position of new node.
94      Raise ValueError if Position p is invalid or p already has a left child.
95      """
96      node = self._validate(p)
97      if node._left is not None: raise ValueError('Left child exists')
98      self._size += 1
99      node._left = self._Node(e, node)                    # node is its parent
100     return self._make_position(node._left)
101
102   def _add_right(self, p, e):
103     """Create a new right child for Position p, storing element e.
104
105     Return the Position of new node.
106     Raise ValueError if Position p is invalid or p already has a right child.
107     """
108     node = self._validate(p)
109     if node._right is not None: raise ValueError('Right child exists')
110     self._size += 1
111     node._right = self._Node(e, node)                   # node is its parent
112     return self._make_position(node._right)
113
114   def _replace(self, p, e):
115     """Replace the element at position p with e, and return old element."""
116     node = self._validate(p)
117     old = node._element
118     node._element = e
119     return old
```

Code Fragment 8.10: Nonpublic update methods for the LinkedBinaryTree class (continued in Code Fragment 8.11).

```
120   def _delete(self, p):
121     """Delete the node at Position p, and replace it with its child, if any.
122
123     Return the element that had been stored at Position p.
124     Raise ValueError if Position p is invalid or p has two children.
125     """
126     node = self._validate(p)
127     if self.num_children(p) == 2: raise ValueError('p has two children')
128     child = node._left if node._left else node._right        # might be None
129     if child is not None:
130       child._parent = node._parent      # child's grandparent becomes parent
131     if node is self._root:
132       self._root = child                # child becomes root
133     else:
134       parent = node._parent
135       if node is parent._left:
136         parent._left = child
137       else:
138         parent._right = child
139     self._size -= 1
140     node._parent = node                 # convention for deprecated node
141     return node._element
142
143   def _attach(self, p, t1, t2):
144     """Attach trees t1 and t2 as left and right subtrees of external p."""
145     node = self._validate(p)
146     if not self.is_leaf(p): raise ValueError('position must be leaf')
147     if not type(self) is type(t1) is type(t2):   # all 3 trees must be same type
148       raise TypeError('Tree types must match')
149     self._size += len(t1) + len(t2)
150     if not t1.is_empty():               # attached t1 as left subtree of node
151       t1._root._parent = node
152       node._left = t1._root
153       t1._root = None                   # set t1 instance to empty
154       t1._size = 0
155     if not t2.is_empty():               # attached t2 as right subtree of node
156       t2._root._parent = node
157       node._right = t2._root
158       t2._root = None                   # set t2 instance to empty
159       t2._size = 0
```

Code Fragment 8.11: Nonpublic update methods for the LinkedBinaryTree class (continued from Code Fragment 8.10).

Performance of the Linked Binary Tree Implementation

To summarize the efficiencies of the linked structure representation, we analyze the running times of the LinkedBinaryTree methods, including derived methods that are inherited from the Tree and BinaryTree classes:

- The len method, implemented in LinkedBinaryTree, uses an instance variable storing the number of nodes of T and takes $O(1)$ time. Method is_empty, inherited from Tree, relies on a single call to len and thus takes $O(1)$ time.
- The accessor methods root, left, right, parent, and num_children are implemented directly in LinkedBinaryTree and take $O(1)$ time. The sibling and children methods are derived in BinaryTree based on a constant number of calls to these other accessors, so they run in $O(1)$ time as well.
- The is_root and is_leaf methods, from the Tree class, both run in $O(1)$ time, as is_root calls root and then relies on equivalence testing of positions, while is_leaf calls left and right and verifies that None is returned by both.
- Methods depth and height were each analyzed in Section 8.1.3. The depth method at position p runs in $O(d_p + 1)$ time where d_p is its depth; the height method on the root of the tree runs in $O(n)$ time.
- The various update methods add_root, add_left, add_right, replace, delete, and attach (that is, their nonpublic implementations) each run in $O(1)$ time, as they involve relinking only a constant number of nodes per operation.

Table 8.1 summarizes the performance of the linked structure implementation of a binary tree.

Operation	**Running Time**
len, is_empty	$O(1)$
root, parent, left, right, sibling, children, num_children	$O(1)$
is_root, is_leaf	$O(1)$
depth(p)	$O(d_p + 1)$
height	$O(n)$
add_root, add_left, add_right, replace, delete, attach	$O(1)$

Table 8.1: Running times for the methods of an n-node binary tree implemented with a linked structure. The space usage is $O(n)$.

8.3.2 Array-Based Representation of a Binary Tree

An alternative representation of a binary tree T is based on a way of numbering the positions of T. For every position p of T, let $f(p)$ be the integer defined as follows.

- If p is the root of T, then $f(p) = 0$.
- If p is the left child of position q, then $f(p) = 2f(q) + 1$.
- If p is the right child of position q, then $f(p) = 2f(q) + 2$.

The numbering function f is known as a ***level numbering*** of the positions in a binary tree T, for it numbers the positions on each level of T in increasing order from left to right. (See Figure 8.12.) Note well that the level numbering is based on *potential* positions within the tree, not actual positions of a given tree, so they are not necessarily consecutive. For example, in Figure 8.12(b), there are no nodes with level numbering 13 or 14, because the node with level numbering 6 has no children.

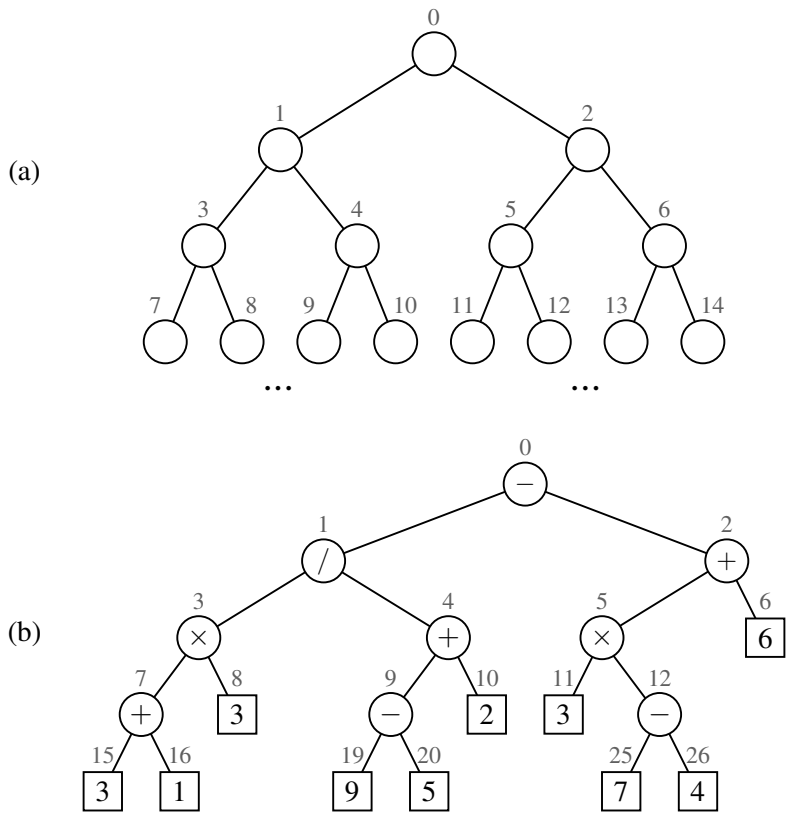


Figure 8.12: Binary tree level numbering: (a) general scheme; (b) an example.

The level numbering function f suggests a representation of a binary tree T by means of an array-based structure A (such as a Python list), with the element at position p of T stored at index $f(p)$ of the array. We show an example of an array-based representation of a binary tree in Figure 8.13.

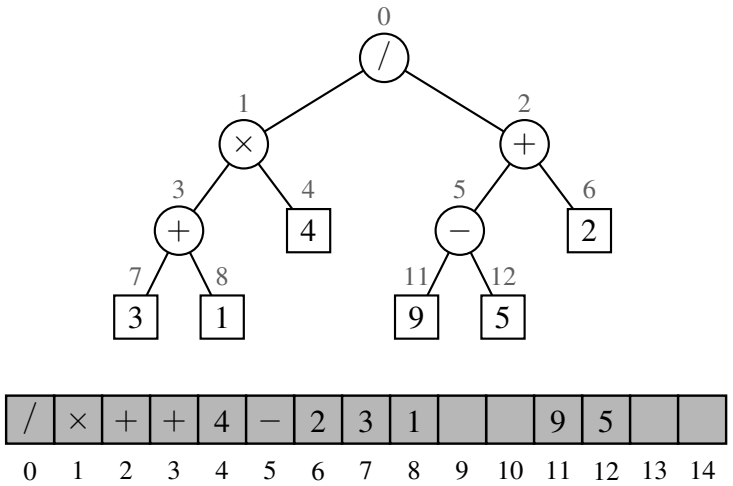


Figure 8.13: Representation of a binary tree by means of an array.

One advantage of an array-based representation of a binary tree is that a position p can be represented by the single integer $f(p)$, and that position-based methods such as root, parent, left, and right can be implemented using simple arithmetic operations on the number $f(p)$. Based on our formula for the level numbering, the left child of p has index $2f(p)+1$, the right child of p has index $2f(p)+2$, and the parent of p has index $\lfloor (f(p)-1)/2 \rfloor$. We leave the details of a complete implementation as an exercise (R-8.18).

The space usage of an array-based representation depends greatly on the shape of the tree. Let n be the number of nodes of T, and let f_M be the maximum value of $f(p)$ over all the nodes of T. The array A requires length $N = 1 + f_M$, since elements range from $A[0]$ to $A[f_M]$. Note that A may have a number of empty cells that do not refer to existing nodes of T. In fact, in the worst case, $N = 2^n - 1$, the justification of which is left as an exercise (R-8.16). In Section 9.3, we will see a class of binary trees, called "heaps" for which $N = n$. Thus, in spite of the worst-case space usage, there are applications for which the array representation of a binary tree is space efficient. Still, for general binary trees, the exponential worst-case space requirement of this representation is prohibitive.

Another drawback of an array representation is that some update operations for trees cannot be efficiently supported. For example, deleting a node and promoting its child takes $O(n)$ time because it is not just the child that moves locations within the array, but all descendants of that child.

8.3.3 Linked Structure for General Trees

When representing a binary tree with a linked structure, each node explicitly maintains fields left and right as references to individual children. For a general tree, there is no a priori limit on the number of children that a node may have. A natural way to realize a general tree T as a linked structure is to have each node store a single *container* of references to its children. For example, a children field of a node can be a Python list of references to the children of the node (if any). Such a linked representation is schematically illustrated in Figure 8.14.

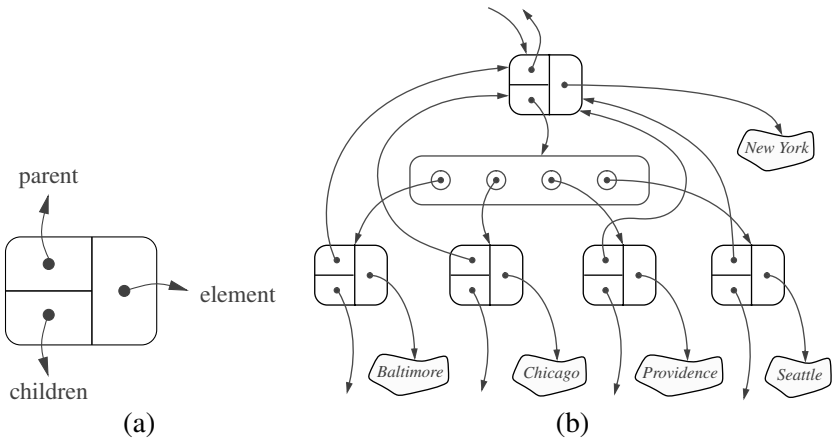


Figure 8.14: The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.

Table 8.2 summarizes the performance of the implementation of a general tree using a linked structure. The analysis is left as an exercise (R-8.14), but we note that, by using a collection to store the children of each position p, we can implement children(p) by simply iterating that collection.

Operation	Running Time
len, is_empty	$O(1)$
root, parent, is_root, is_leaf	$O(1)$
children(p)	$O(c_p+1)$
depth(p)	$O(d_p+1)$
height	$O(n)$

Table 8.2: Running times of the accessor methods of an n-node general tree implemented with a linked structure. We let c_p denote the number of children of a position p. The space usage is $O(n)$.